CST2550 Software Engineering Management and Development
TV Programme Scheduling System
Group Coursework - Project Management Report
Submission Date: 10th April 2026

Team Leader (Scrum Master)	Khrisvin	Nepaulsing	M01003462
Secretary	Veedoorsingh	Gopal	M01013795
Developer	Mehdi	Bhoyroo	M00983496
Developer	Navneet	Sookraz	M01000843
Tester	Pranay	Poheith	M01013276

Contents

Project Description 1

Team Roles and Responsibilities 1

Tools and Collaboration Platforms 1

Meeting Management and Communication 2

Workload Division 2

Project Planning 3

Planning Approach 3

Work Breakdown Structure (WBS) 3

Figure 1. Work Breakdown Structure (WBS) of the TV Programme Scheduling System project. 4

Work Timeline 4

Gantt Chart 5

Figure 2. Gantt chart of the TV Programme Scheduling System project timeline. 5

Project Design and Implementation Management 5

Activity Diagram 5

Figure 3. Activity diagram of the main scheduling workflow. 6

Flowchart 6

Figure 4. Flowchart of the schedule management logic. 7

Use Case 7

Figure 5. Use case diagram of the main system interactions 7

Testing Management 7

Risk Management 8

Conclusion 8

Project Description

The TV Programme Scheduling System was developed to help users organise television programmes more efficiently through a desktop application built in C#. The system supports role-based access for Admin, Manager, Client, and Viewer, and includes features such as adding and removing schedules, viewing schedules by channel, suggesting available slots, managing programmes, and using an integrated AI assistant for schedule-related support.

From a management perspective, the project had two linked objectives. The first was to deliver a working software product that met the functional requirements of the module. The second was to manage the work in a structured way so that planning, communication, testing, and final integration were completed on time.

The scope of the project included requirements discussion, interface design, implementation in Visual Studio, SQL database setup, version control through GitHub, diagram preparation in draw.io, testing, and report preparation. The team therefore had to balance technical work with coordination and documentation throughout the development period.

Team Roles and Responsibilities

A five-member team was formed for the project. Roles were allocated so that each member had a clear area of responsibility while still being able to support the wider development process. This helped reduce confusion, improved accountability, and made progress easier to monitor.

Team Member	Role	Responsibilities
Khrisvin Nepaulsing	Team Leader (Scrum Master)	Coordinated the project, organised meetings, tracked task progress, followed up on deadlines and supported final integration.
Veedoorsingh Gopal	Secretary	Recorded meeting notes, documented decisions, helped maintain communication records and supported report organisation.
Mehdi Bhoyroo	Developer	Worked on implementation tasks, supported feature development and helped with code integration and issue fixing.
Navneet Sookraz	Developer	Worked on implementation tasks, supported interface and logic development, and contributed code through GitHub.
Pranay Poheith	Tester	Prepared and ran tests, reported defects, checked outputs against expected behaviour and supported verification before submission.

The chosen roles reflected the needs of the project. A Scrum Master role was useful because the team needed one member to keep work moving and make sure responsibilities were clear. The Secretary role supported documentation and decision tracking, which was important for the project management report. The two developers focused mainly on implementation tasks, while the tester concentrated on checking whether the system behaved correctly. Even with defined roles, members still communicated regularly and supported one another when clarification or integration work was needed.

Tools and Collaboration Platforms

The team used a small set of practical tools during the project. Each tool was selected because it directly supported either development, communication, documentation or coordination.

Tool	Purpose	Why it was useful
Visual Studio	Main development environment	Allowed the team to write, run and debug the system in one place.
GitHub	Version control and code sharing	Made it easier to manage updates, keep a shared code base and monitor contributions.
SQL Server LocalDB / SQL tools	Database setup and testing	Supported schema creation, data persistence, and query checking during development.
draw.io	Diagrams and visual planning	Used to prepare system diagrams and other report visuals in a clear format.
WhatsApp	Communication and quick updates through group calls and messages	Supported rapid discussion when the team was not in the same location.

GitHub was especially important because it provided a shared repository and helped the team manage changes more safely. Visual Studio supported the practical development side, while SQL LocalDB supported database setup and persistence testing. draw.io was useful when the group needed diagrams for both the technical and project management reports. WhatsApp was not a development tool, but it was central to coordination because it allowed the team to organise calls and keep quick communication active outside formal meetings.

Meeting Management and Communication

Communication was handled through a combination of face-to-face meetings and WhatsApp group calls. The team held meetings both at home and at the university, depending on availability. This flexible approach was practical because it allowed the members to continue discussing the project even when they were not all in the same place.

Face-to-face meetings were mainly used for planning, reviewing progress, discussing technical issues and dividing work. WhatsApp calls were used for follow-up discussions, short updates and urgent clarifications. This helped the team maintain regular contact without relying only on one communication method.

Communication method	How it was used	Benefit
Meetings at university	Planning, reviewing progress, checking assigned tasks	Useful for direct discussion and faster group decisions.
Meetings at home	Longer working sessions, integration and report discussion	Allowed focused collaboration in a practical setting.
WhatsApp group calls/messages	Quick updates, clarifications and scheduling discussions	Kept communication active between meetings.

Meeting notes were mainly supported by the secretary, while key decisions were also reinforced through group discussion. This created a simple but effective communication structure. Although the team did not depend on complex project management software, the combination of meetings, calls and role allocation helped maintain organised collaboration.

Workload Division

Work was divided according to role and practical strengths. This was important because the project involved both technical tasks and coordination tasks. Assigning responsibilities clearly reduced duplication of effort and made progress easier to measure.

Area of work	Person(s) responsible	Main tasks
Project coordination	Khrisvin	Monitoring progress, organising meetings, and checking completion of planned tasks.
Documentation and meeting records	Veedoorsingh	Recording notes, helping organise report material and decisions.
Development	Mehdi and Navneet	Implementing features, editing code, supporting integration.
Testing and verification	Pranay	Preparing test cases, checking outputs, reporting defects and confirmations.
Shared report preparation	All members with coordination support	Reviewing content, diagrams and consistency before submission.

The workload was therefore divided in a responsibility-based way rather than randomly. This made the project more manageable because each person knew what was expected. At the same time, members still discussed issues together during meetings, which helped when tasks overlapped or when one area affected another, such as implementation and testing.

Project Planning

The team used a simple Agile-style approach. Instead of trying to complete the whole project at once, the work was divided into smaller implementation-based stages such as user access, programme and schedule features, database work, AI integration, testing, and documentation. This made the project easier to organise and allowed problems to be identified earlier.

Planning Approach

The planning approach was practical rather than highly formal. The team first identified the core system components, including user access, programme management, schedule management, database work, and AI integration, and then linked these to responsibilities. This allowed the group to focus on essential implementation work before moving to testing, refinement, and submission materials. The Scrum Master monitored whether the work remained aligned with deadlines and whether tasks needed to be redistributed.

Work Breakdown Structure (WBS)

The Work Breakdown Structure (WBS) was used to break down the work into implementation-based work packages that reflected the main parts of the TV Programme Scheduling System. These included user access and interfaces, programme and schedule features, the custom data structure, database implementation, AI assistant integration, testing, and documentation. This made the project easier to organise and helped the team assign tasks according to the actual system components being developed.

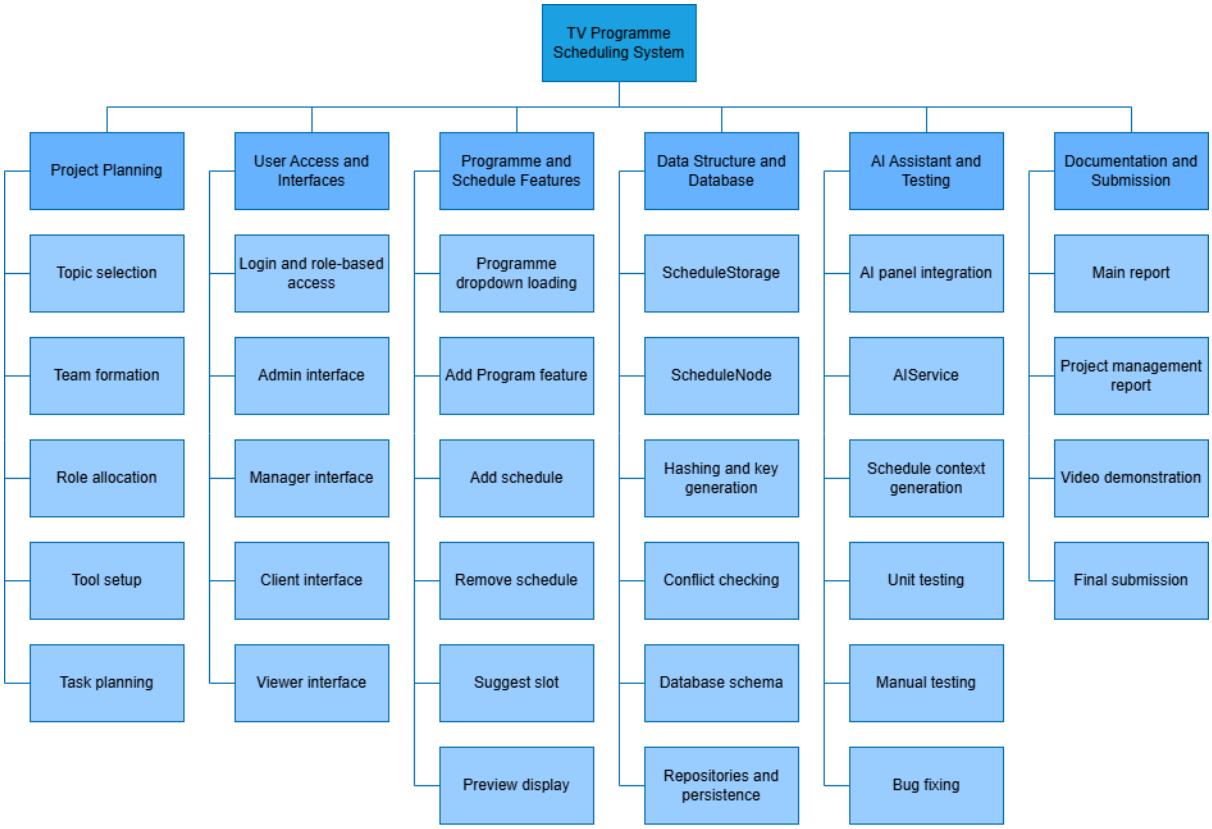

Figure 1. Work Breakdown Structure (WBS) of the TV Programme Scheduling System project.

Work Timeline

The timeline was used to organise the order in which the main project activities were carried out. Although some flexibility was needed because of timetable constraints and changing availability, the overall sequence remained stable and supported steady progress from planning to final submission.

Stage	Main activities	Responsible members	Expected outcome
Weeks 1-2	Project discussion, role allocation, requirement review, tool setup	All members	Shared understanding of project scope and working setup established
Weeks 2-3	Initial system design, interface planning, database design, diagrams	Team leader, developers, secretary	Clear system structure and design decisions prepared
Weeks 3-6	Implementation of login, user roles, programme management, schedule management, and interfaces	Developers with team support	Core application features developed
Weeks 5-6	Custom data structure and database integration	Developers	Schedule storage and SQL persistence completed
Weeks 7-8	AI assistant integration, testing, debugging, and issue fixing	Developers, tester	Main functionality refined and verified
Weeks 8-10	Report writing, diagrams, video preparation, final review, and submission	All members	Submission materials completed and finalised

Although the timeline was simplified, it helped the team focus on the main priorities at each stage of the project. When minor delays occurred, communication through meetings and WhatsApp allowed the team to adjust tasks without losing the overall direction of the work.

Gantt Chart

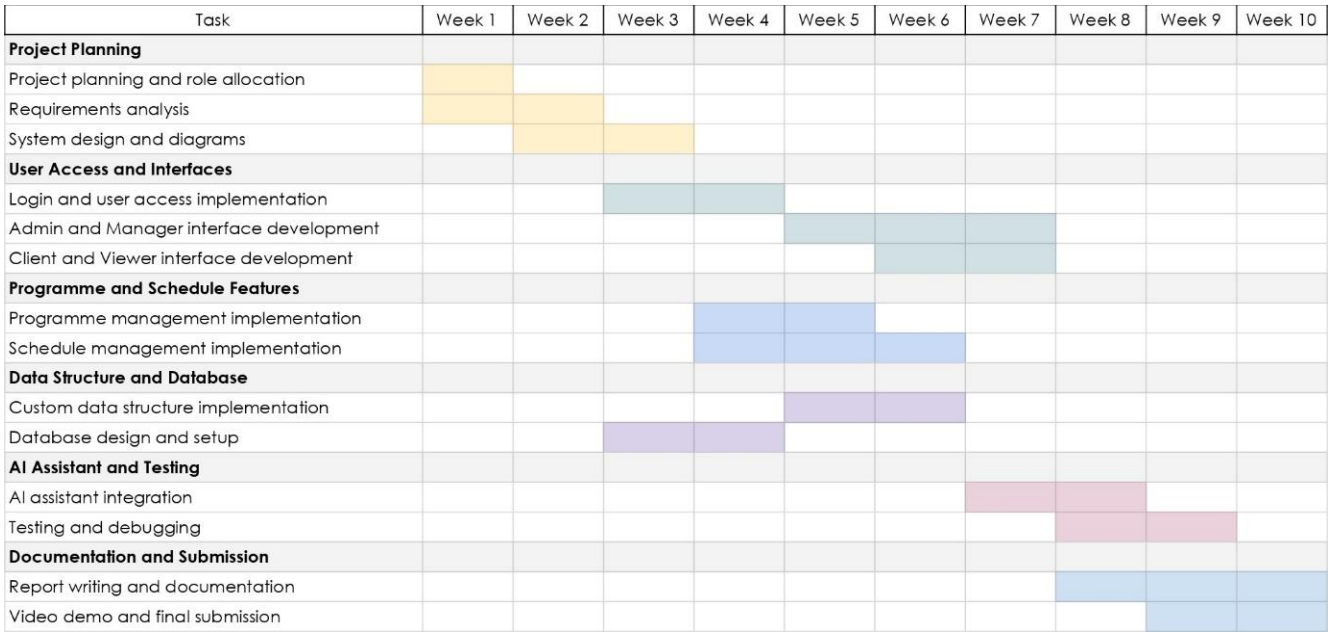

Figure 2. Gantt chart of the TV Programme Scheduling System project timeline.

Project Design and Implementation Management

Managing implementation required continuous coordination between planning, coding, and review. The team needed to make sure that design decisions were understood before or during implementation so that the software remained consistent across user access, programme management, schedule features, database integration, and AI support.

Visual representations created in draw.io supported this process by helping members discuss the system more clearly. Diagrams were useful because they reduced ambiguity and gave the team a shared point of reference during both feature development and report writing.

Implementation tasks were mainly handled by the developers, but progress was tracked through team discussions. The Scrum Master checked whether assigned tasks were advancing, while the Secretary helped ensure that decisions were not lost between meetings. This coordination process reduced the risk of members working in isolation and made final integration more manageable.

Activity Diagram

The diagram shows how a user interacts with the TV Programme Scheduling System from start to end. The user can add a schedule, remove a schedule, view schedules by channel, or ask the system to suggest the next free slot. Decision points are used to validate input, check whether schedules exist, identify time conflicts, and determine whether a free slot is available. After each action, the flow returns to the main menu until the user chooses to exit the system.

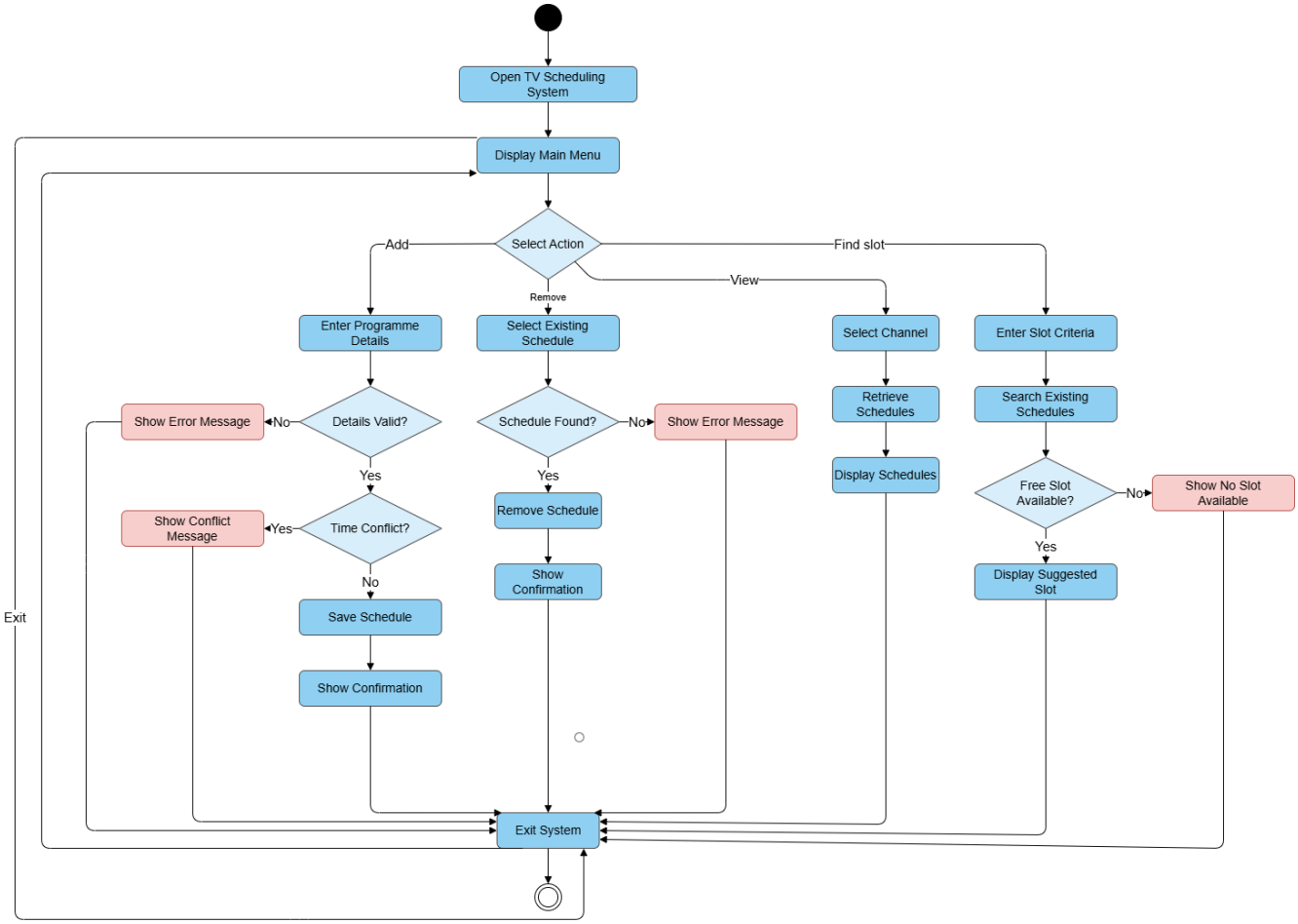

Figure 3. Activity diagram of the main scheduling workflow.

Flowchart

This flowchart shows the main system flow for the TV Scheduling System. The user opens the application, selects an action from the main menu, and the system follows different paths based on the selected function. Decision shapes are used to validate input, detect time conflicts, check whether a schedule exists, and determine whether a free slot is available. The flowchart covers the main features mentioned in the report: add schedule, remove schedule, view schedules by channel, and suggest next free slot.

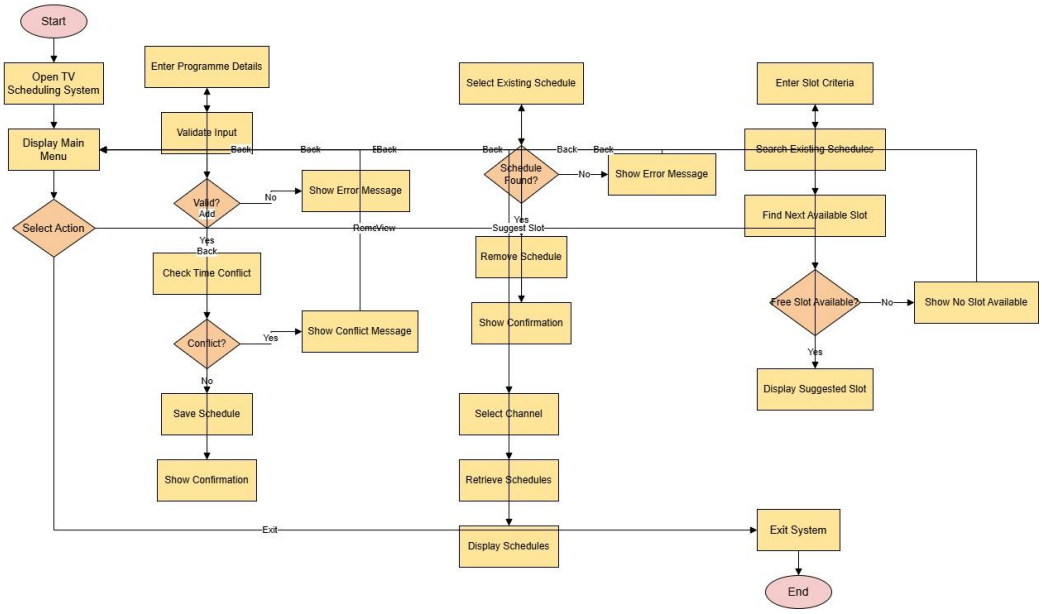

Figure 4. Flowchart of the schedule management logic.

Use Case

The main actor is the Scheduling User. The user can add a schedule, remove a schedule, view schedules by channel, and request the next available free slot. Some use cases include internal system functions such as input validation, conflict checking, retrieving schedule data, and searching for free slots. The rectangle shows the system boundary for the TV Programme Scheduling System.

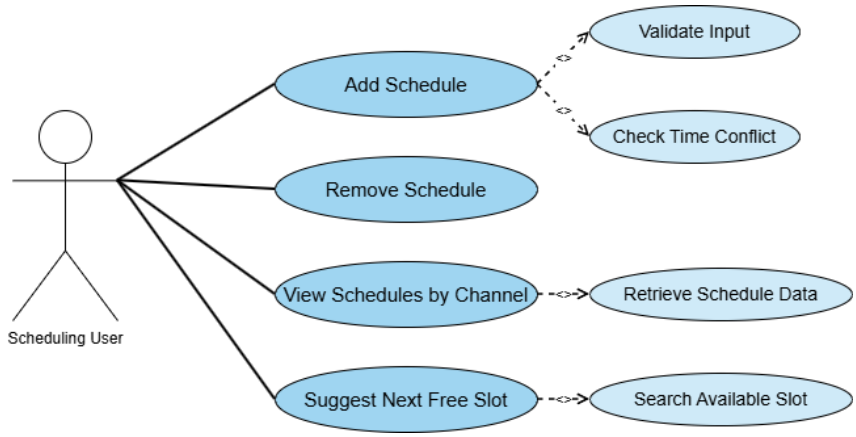

Figure 5. Use case diagram of the main system interactions.

Testing Management

Testing management was mainly led by the team tester, Pranay Poheith, with support from the developers when issues were found. The purpose of testing was to confirm that the system behaved as expected and that the main features could be completed correctly.

The main areas checked included login and role-based access, adding and removing schedules, viewing schedules, programme management, and slot suggestion. Valid cases were important, but the team also needed to observe invalid or problematic cases, such as conflicts, missing information, failed login attempts, and unexpected outputs.

Test area	Example check	Reason for test	Outcome
Login and role access	Check that valid and invalid logins behave correctly	Confirms user access control works	Verified during testing stage
Add schedule	Check that a valid schedule can be saved	Confirms a core system feature works	Verified during testing stage
Remove schedule	Check that a selected schedule can be removed	Confirms existing data can be updated	Verified during testing stage
Programme management	Check that a new programme can be added and loaded into the dropdown	Confirms programme features work correctly	Verified during testing stage
Suggest slot	Check that the system suggests an available time	Supports conflict avoidance and decision support	Verified during testing stage

When problems were identified, the tester communicated them back to the team so that corrections could be made. This feedback loop was important because it linked testing directly to improvement rather than treating testing as a separate final step only.

Risk Management

Risk management was necessary because software projects can face both technical and coordination problems. The team, therefore, needed to think not only about coding risks but also about communication and deadline risks.

Risk	Potential impact	Mitigation approach
Communication gaps	Tasks may be delayed or duplicated	Regular meetings and WhatsApp communication kept members updated
Integration issues	Different parts may not work together smoothly	GitHub, shared reviews, and staged integration reduced this risk
Database issues	Stored data may not load, save, or match the application correctly	LocalDB testing and repository checks supported database reliability
AI service problems	External AI features may fail because of configuration or authentication issues	The team tested the AI feature separately and retained core schedule logic in the application
Testing delays	Errors may remain close to submission	A dedicated tester role helped keep verification visible
Deadline pressure	Quality may fall if work is left too late	Task division and follow-up by the Scrum Master supported earlier completion

These risks were managed with simple but practical methods. The team did not use a formal enterprise risk framework, but regular communication, clear roles, shared tools, and staged testing reduced uncertainty and helped problems to be addressed quickly.

Conclusion

Overall, the project was managed through clear role allocation, practical communication and a simple Agile-style planning approach. The team used face-to-face meetings, WhatsApp communication, Visual Studio, GitHub, Windows Prompt and draw.io to support development and coordination. This

combination of tools and roles helped the group produce a structured workflow around the TV Programme Scheduling System.

The project management approach was effective because responsibilities were defined, progress was monitored and testing was given a clear place in the process. While the management structure remained simple, it was appropriate for the scale of the project and supported completion of the main objectives within the module context.